

Week 1: Requirement Analysis and Customer Database Module

1. Requirement Analysis

Understanding the Organic Grocery Store Management System

The Organic Grocery Store Management System is a database-based application used to manage the daily activities of an organic grocery store. Customers can view organic products, select required items, place orders, make payments, and receive deliveries. The database maintains important information about customers, products, suppliers, orders, inventory, payments, and deliveries. Properly organized data helps the store manage its operations efficiently and reduces manual work.

The Process Flow

- **Customer Registration:** The customer creates an account by providing basic personal details.
- **Product Search:** The customer searches and views available organic grocery products.
- **Order Placement:** The customer selects products and places an order.
- **Payment:** The customer completes the payment using an available payment method.
- **Inventory Update:** The product stock is automatically updated after the order is placed.
- **Order Processing:** The order status is updated as Pending, Packed, Shipped, or Delivered.
- **Delivery:** The order is assigned to a delivery person and delivered to the customer.
- **Reports:** Sales, inventory, customer orders, and business summaries can be generated.

Entities

Entity	Description	Key Attributes	Key Relationships
Customer	Stores information about customers who purchase organic products.	Customer_id (PK), first_name, last_name, email, phone, address	One-to-Many with Order

Entity	Description	Key Attributes	Key Relationships
Product	Contains details of organic grocery products available for sale.	Product_id (PK), product_name, category, price, stock_quantity, supplier_id (FK)	Many-to-One with Supplier, One-to-Many with Order
Supplier	Stores information about suppliers who provide products to the store.	Supplier_id (PK), supplier_name, email, phone, address	One-to-Many with Product
Order	Records purchases made by customers.	Order_id (PK), customer_id (FK), order_date, status, total_amount	Many-to-One with Customer, One-to-One with Payment
Payment	Maintains payment information related to customer orders.	Payment_id (PK), order_id (FK), payment_method, status, amount, payment_date	One-to-One with Order
Inventory	Maintains information about product stock availability.	Inventory_id (PK), product_id (FK), quantity, reorder_level	One-to-One with Product
Delivery	Stores delivery information and delivery status.	Delivery_id (PK), order_id (FK), delivery_person, delivery_status, delivery_date	One-to-One with Order

Customer Table Creation

The **Customer** table is created to maintain the basic details of customers registered in the organic grocery store system. A unique **Customer_id** is used to identify each customer record.

Customer Table

```
SQL> CREATE TABLE Customer (customer_id INT PRIMARY KEY,first_name VARCHAR(50), last_name VARCHAR(50),email VARCHAR(100),phone VARCHAR(15),address VARCHAR(200));
```

Table created.

```
SQL> DESCRIBE Customer;
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER(38)
FIRST_NAME		VARCHAR2(50)
LAST_NAME		VARCHAR2(50)
EMAIL		VARCHAR2(100)
PHONE		VARCHAR2(15)
ADDRESS		VARCHAR2(200)

Insert Values

The **INSERT** command is used to add customer information into the Customer table.

Multiple customer records can be entered into the database.

```
SQL> INSERT INTO Customer(customer_id, first_name, last_name, email, phone, address)VALUES(1, 'Arun', 'Kumar', 'arun@gmail.com', '9876543210', 'Chennai');
```

1 row created.

```
SQL> INSERT INTO Customer(customer_id, first_name, last_name, email, phone, address)VALUES(2, 'Priya', 'Raj', 'priya@gmail.com', '9876543211', 'Coimbatore');
```

1 row created.

```
SQL> INSERT INTO Customer(customer_id, first_name, last_name, email, phone, address)VALUES(3, 'Kavin', 'M', 'kavin@gmail.com', '9876543212', 'Madurai');
```

1 row created.

Display Customer Records

The stored customer information can be viewed using the **SELECT** command.

```
SQL> SELECT * FROM Customer;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE	ADDRESS
1	Arun	Kumar	arun@gmail.com	9876543210	Chennai
2	Priya	Raj	priya@gmail.com	9876543211	Coimbatore
3	Kavin	M	kavin@gmail.com	9876543212	Madurai

Update Customer Information

Existing information in the Customer table can be modified using the **UPDATE** statement.

The **WHERE** condition is used to identify the particular customer record that needs to be changed.

```
SQL> UPDATE Customer SET phone = '9999999999' WHERE customer_id = 1;  
1 row updated.
```